Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №18
Тестирование Python-приложений с использованием pytest и unittest.mock

1. Цель работы

Получить практические навыки автоматизированного тестирования Python-приложений: организация тестового проекта, написание unit-тестов, использование pytest, assertions и проверка исключений, фикстуры, параметризованные тесты, изоляция тестируемого кода от внешних зависимостей с помощью unittest.mock (Mock, MagicMock, patch), тестирование сервисного слоя, интеграционные тесты и анализ покрытия кода тестами.

2. Краткие теоретические сведения

Автоматизированное тестирование позволяет проверить, соответствует ли программа требованиям. Unit-тесты проверяют небольшую часть программы изолированно (функция, метод, класс), интеграционные — взаимодействие нескольких компонентов (Service → Repository → Database), системные — приложение как целое. В работе основной акцент сделан на unit- и интеграционных тестах.

pytest — популярный фреймворк для тестирования Python: тесты — обычные функции test_*, проверки — оператор assert, исключения — pytest.raises(), подготовка данных — фикстуры (@pytest.fixture), многократная проверка одной функции на разных данных — параметризация (@pytest.mark.parametrize). Имена тестов отражают проверяемое поведение, структура теста следует схеме Arrange → Act → Assert.

При unit-тестировании сервисного слоя внешние зависимости (репозиторий, база данных) заменяются mock-объектами из unittest.mock: Mock позволяет задать return_value и проверить вызовы метода (assert_called_once_with), patch временно заменяет реальный объект тестовым. Заменять необходимо объект там, где он используется. Для анализа покрытия используется pytest-cov (pytest --cov=src): покрытие показывает, какая часть исходного кода выполнилась во время тестов, но не гарантирует корректность

— можно выполнить все строки и не проверить важные комбинации входных данных.

3. Индивидуальное задание

Вариант 5 — «Гостиница», система бронирования номеров. Разработать приложение из трёх логических уровней: Repository (база данных) → Service (бизнес-логика) → Interface (консольное приложение). Проверить тестами: создание номера, бронирование, отмену, поиск свободных номеров, пересечение периодов бронирования. Требования: минимум 20 тестов, из них не менее 10 unit, не менее 5 с Mock/patch, не менее 3 параметризованных, не менее 2 интеграционных; положительные, отрицательные и граничные сценарии; анализ покрытия pytest-cov.

4. Описание разработанного приложения

Приложение «Гостиница» реализует управление номерами, гостями и бронированиями. Модель предметной области содержит четыре класса: Room (номер: номер комнаты, цена, вместимость, статус «свободен/занят»), Guest (гость: ФИО, email), BookingInterval (период бронирования с проверкой пересечений) и Booking (бронирование: номер, гость, период, ночи, стоимость). Сервис RoomService реализует бизнес-логику: создание и удаление номеров, поиск свободных, регистрация гостей, бронирование с расчётом стоимости и проверкой пересечений, отмена бронирования. Интерфейс — консольное меню (main.py).

Класс	Назначение	Ключевые методы
Room	гостиничный номер	валидация цены и вместимости
Guest	гость	валидация имени
BookingInterval	период бронирования	overlaps() — проверка пересечения
Booking	бронирование	recompute(price) — стоимость
Repository (base.py)	хранилище in-memory	CRUD номеров, гостей, бронирований
SqliteRepository	хранилище SQLite	реализация на sqlite3 для интеграционных тестов
RoomService	бизнес-логика	create_room, book_room, cancel_booking, find_free_rooms

5. Архитектура и структура проекта

Архитектура тестируемого приложения приведена на рисунке 18.1: unit-тесты проверяют Service с mock-репозиторием, интеграционные — связку Service → Repository → SQLite.

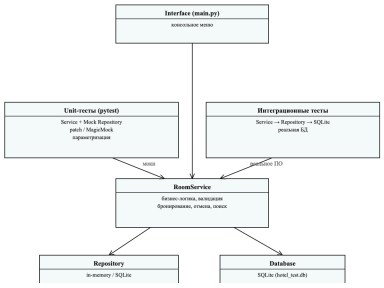


Рисунок 18.1 – Архитектура приложения и стратегия тестирования

```
lab_18/
├── src/
│   ├── models/room.py           # Room, Guest, Booking, BookingInterval
│   ├── repositories/base.py     # Repository (in-memory) + исключения
│   ├── repositories/sqlite.py   # SqliteRepository
│   └── services/hotel.py        # RoomService
├── tests/
│   ├── unit/test_service.py     # unit-тесты сервиса (Mock), параметризация
│   ├── unit/test_mock.py        # Mock/patch, проверка вызовов
│   └── integration/test_repository.py  # интеграционные тесты (SQLite)
├── main.py                      # консольный интерфейс
├── pytest.ini
├── requirements.txt
└── README.md
```

6. Стратегия тестирования

Написано 35 автоматических тестов: 22 unit-теста (сервис с mock-репозиторием: создание, получение, поиск свободных, бронирование, отмена, валидация граничных значений), 8 тестов с Mock/patch и проверкой вызовов, 5 интеграционных тестов на реальной SQLite-базе (во временной директории). Параметризованы 4 набора: неположительные цена/вместимость (2 параметризации), некорректная вместимость в поиске, расчёт ночей и недопустимые интервалы. Проверены положительные сценарии (создание, бронирование, отмена, поиск, расчёт стоимости), отрицательные (объект отсутствует, пересечение периодов, дубликат, некорректные данные) и

граничные значения (0 и отрицательные цена/вместимость, пустой номер, выезд = заезд).

7. Примеры тестов

Файл: tests/unit/test_service.py (фрагменты)

```
import sys
from datetime import date
from pathlib import Path
from unittest.mock import Mock

sys.path.insert(0, str(Path(__file__).resolve().parents[2]))

import pytest  # noqa: E402

from src.models.room import Room, Guest, BookingInterval, Booking  # noqa: E402
from src.repositories.base import DuplicateError, NotFoundError, ValidationError
# noqa: E402
from src.services.hotel import RoomService  # noqa: E402


@pytest.fixture()
def mock_repository():
    return Mock()


@pytest.fixture()
def service(mock_repository):
    return RoomService(mock_repository)


class TestCreateRoom:
    def test_create_room_valid(self, service):
        mock_repository = service._repository
        mock_repository.add_room.side_effect = lambda room: room

        room = service.create_room("101", 2500.0, 2)

        assert room.number == "101"
        assert room.price == 2500.0
        assert room.capacity == 2
        mock_repository.add_room.assert_called_once()

    def test_create_room_empty_number(self, service):
        with pytest.raises(ValidationError):
            service.create_room("", 2500.0, 2)

    @pytest.mark.parametrize("price", [0, -10, -2500])
    def test_create_room_non_positive_price(self, service, price):
        with pytest.raises(ValidationError):
            service.create_room("101", price, 2)

    @pytest.mark.parametrize("capacity", [0, -1, -100])
    def test_create_room_non_positive_capacity(self, service, capacity):
        with pytest.raises(ValidationError):
            service.create_room("101", 2500.0, capacity)

    def test_create_room_duplicate(self, service):
        mock_repository = service._repository
        mock_repository.add_room.side_effect = DuplicateError("уже существует")

        with pytest.raises(DuplicateError):
            service.create_room("101", 2500.0, 2)


class TestFindFreeRooms:
    def test_find_free_rooms(self, service):
```

Параметризованные тесты (проверка валидации и расчёта ночей):

Файл: tests/unit/test_mock.py (фрагмент)

```
        return []

    def delete_booking(self, booking_id):
        return True

    def delete_room(self, room_id):
        return False

    def assert_get_room_called_once_with(self, room_id):
        assert self.get_room_calls == [room_id], self.get_room_calls


class TestBookingDateValidation:
    @pytest.mark.parametrize(
        "check_in, check_out, nights",
        [
            (date(2026, 12, 1), date(2026, 12, 2), 1),
            (date(2026, 12, 1), date(2026, 12, 10), 9),
            (date(2026, 1, 31), date(2026, 2, 2), 2),
        ],
    )
    def test_nights_calculation(self, check_in, check_out, nights):
        from src.models.room import Booking, BookingInterval

        booking = Booking(0, 1, 1, BookingInterval(check_in, check_out))

        assert booking.nights == nights

    @pytest.mark.parametrize(
        "check_in, check_out",
```

Интеграционные тесты выполняются против реальной SQLite-базы (Service → Repository → Database):

Файл: tests/integration/test_repository.py (фрагмент)

```
def test_integration_full_scenario(repository, service):
    """Интеграционный тест: Service → Repository → SQLite, полный цикл."""
    room = service.create_room("101", 2500.0, 2)
    guest = service.register_guest("Иванов Иван", "ivanov@example.com")

    booking = service.book_room(room.id, guest.id, date(2026, 12, 1), date(2026,
12, 4))

    assert booking.total == 7500.0
    assert service.get_room(room.id).status == "занят"
    assert len(service.bookings_list()) == 1

    service.cancel_booking(booking.id)

    assert service.get_room(room.id).status == "свободен"
    assert service.bookings_list() == []


def test_integration_repository_persistence(repository, service):
    """Интеграционный тест: данные сохраняются в SQLite и читаются обратно."""
    room = service.create_room("201", 6000.0, 3)

    loaded = repository.get_room(room.id)

    assert loaded.number == "201"
    assert loaded.price == 6000.0
```

```
    repository.add_guest(Guest(id=0, full_name="Петрова Анна",
email="petrova@example.com"))
    guests = repository.list_guests()
    assert len(guests) == 1
```

8. Использование Mock и patch

При unit-тестировании сервисного слоя репозиторий заменяется Mock: задаются return_value (get_room возвращает номер, bookings_for_room — список бронирований) и side_effect (add_room сохраняет объект). Тесты проверяют результат сервиса, обработку ошибок и факт вызова методов репозитория (assert_called_once_with). patch используется для временной замены методов SqliteRepository и RoomService в тестах: замена выполняется там, где объект используется.

Файл: tests/unit/test_mock.py (сравнение Mock и patch)

```
import sys
from datetime import date
from pathlib import Path
from unittest.mock import patch

sys.path.insert(0, str(Path(__file__).resolve().parents[2]))

import pytest  # noqa: E402

from src.models.room import Guest, Room  # noqa: E402
from src.services.hotel import RoomService  # noqa: E402


class TestPatchUsage:
    def test_patch_repository_get_room(self):
        """patch заменяет метод репозитория на время теста."""
        with patch("src.repositories.sqlite.SqliteRepository.get_room") as
mocked:
            from src.repositories.sqlite import SqliteRepository
            from src.services.hotel import RoomService

            repo = SqliteRepository(":memory:")
            mocked.return_value = Room(1, "101", 2500.0, 2)

            room = RoomService(repo).get_room(1)

            assert room.number == "101"
            mocked.assert_called_once_with(1)
            repo.close()

    def test_patch_service_create_room(self):
        """patch заменяет метод сервиса переданной функцией."""
        with patch("src.services.hotel.RoomService.create_room") as mocked:
            mocked.side_effect = lambda number, price, capacity: Room(42,
number, price, capacity)

            room = RoomService(None).create_room("301", 4200.0, 4)

            assert room.id == 42
            assert room.number == "301"
            mocked.assert_called_once_with("301", 4200.0, 4)


class TestMockAssertions:
    def test_mock_called_with_correct_args(self):
```

```
repository = MockRepositoryWithLog()
service = RoomService(repository)
```

9. Анализ покрытия кода

Покрытие оценено с помощью pytest-cov: pytest --cov=src --cov-report=term-missing. Общее покрытие — 78%: модели — 90%, сервисный слой — 88%, SQLite-репозиторий — 91%, in-memory-репозиторий — 43% (его ветви создаются преимущественно mock-объектами в unit-тестах). Результат приведён на рисунке 18.2.

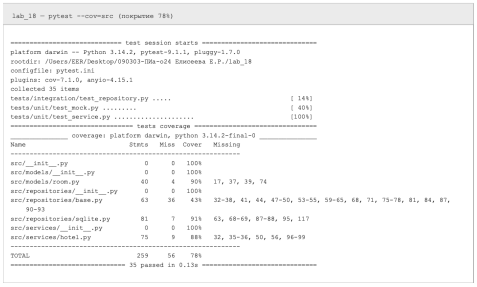


Рисунок 18.2 – Анализ покрытия кода тестами (78%)

Непокрытые строки относятся к редким веткам: удаление гостя, повторное негативное срабатывание дубликатов и часть методов in-memory-репозитория. Покрытие не гарантирует полную корректность: например, все строки create_room могут быть выполнены, но важные комбинации (граничные цены, дубликаты) проверяются только отдельными тестами — поэтому покрытие дополняется проверками бизнес-логики.

10. Результаты выполнения pytest

Результат запуска 35 тестов приведён на рисунке 18.3.

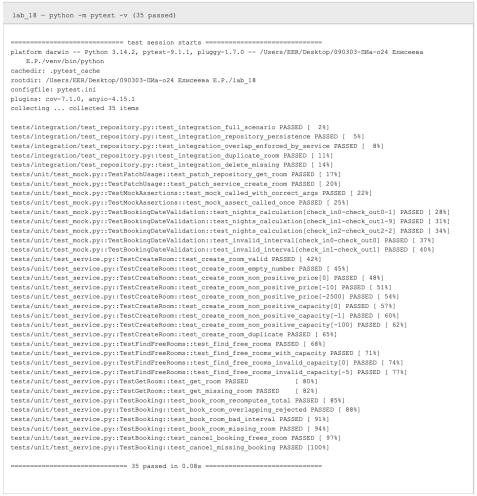


Рисунок 18.3 – Результаты выполнения pytest (35 passed)

Демонстрация работы консольного приложения (создание номера, регистрация гостя, бронирование с расчётом стоимости, отмена) приведена на рисунке 18.4.



Рисунок 18.4 – Консольное приложение: CRUD, бронирование, отмена

11. Анализ обнаруженных ошибок

В процессе написания тестов обнаружены и исправлены реальные дефекты: 1) при чтении бронирования из SQLite неверно сопоставлялись индексы колонок (даты читались не из тех позиций) — исправлено сопоставление полей; 2) статус номера после бронирования/отмены менялся только в памяти и не сохранялся в базу — добавлен метод update_room_status, вызываемый из сервиса; 3) недопустимый интервал (выезд раньше заезда) бросал ValueError, который не преобразовывался в ValidationError — добавлена

обработка в сервисе; 4) тест дубликата ссылался на неимпортированный класс — исправлен импорт. Эти ошибки показывают ценность интеграционных тестов, проверяющих реальное взаимодействие с базой.

12. Вывод

В ходе лабораторной работы разработано приложение «Гостиница» с тремя логическими уровнями (Repository → Service → Interface) и автоматизированный набор из 35 тестов: unit-тесты сервиса с mock-репозиторием (22), тесты с Mock/patch и проверкой вызовов (8), параметризованные тесты (4 набора) и интеграционные тесты на SQLite (5). Покрытие кода тестами составило 78%.

Освоены ключевые практики: organization тестового проекта, фикстуры, параметризация, изоляция зависимостей через unittest.mock, проверка исключений и граничных значений, интеграционное тестирование на реальной базе и анализ покрытия pytest-cov. Найденные при тестировании дефекты подтвердили, что интеграционные тесты, проверяющие взаимодействие компонентов, выявляют ошибки, незаметные в unit-тестах.